

Disciplina:

ORQUESTRAÇÃO DE WORKFLOWS

Professor: Reinaldo Carlos Mendes



PREFECT

AULA 04

Orquestração Moderna com Prefect
A alternativa Pythonic ao Airflow

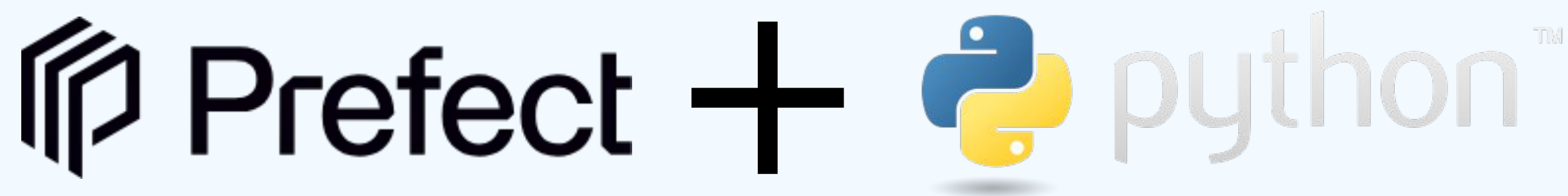


Objetivos de Aprendizagem

- Ao final desta aula, você será capaz de:
 - Compreender as diferenças entre Prefect vs Airflow: filosofia, vantagens e trade-offs
 - Criar um flow Prefect usando decoradores `@flow` e `@task` de forma Pythonic
 - Implementar deployments com schedules e work pools
 - ~~○ Usar Blocks para armazenar credenciais reutilizáveis~~
 - Criar workflows com geração dinâmica usando `.map()`
 - Implementar Automations para workflows event-driven
 - Integrar Prefect com sistemas externos (APIs, webhooks, file systems)

PREFECT

Prefect é uma engine de orquestração open-source, que torna funções Python em pipelines de dados de nível de produção. Permite construir e agendar seus workflows usando puramente Python.



PREFECT

- ***O problema do Airflow (segundo Prefect):***
 - Tudo é um **DAG** — conceito confuso para iniciantes
 - Muita "**plumbing**" — BaseOperator, XCom, hooks
 - Código é complexo — operadores customizados exigem herança
- ***Prefect resolve:***
 - **Flows** são `funções Python @flow` — sem DSL especial
 - **Tasks** são `decoradores @task` — sem classes
 - Dados fluem naturalmente — O retorno da task vira o input da próxima (sem necessidade de XCom)
 - Mais Pythonic — sente como código, não como framework

PREFECT: Install Mode

`pip` - Gerenciador de Pacotes

```
pip install -U prefect
```

To confirm that Prefect was installed successfully, run:

```
prefect version
```

`uv` - Gerenciador de Projetos

```
uv pip install prefect

# add prefect to your pyproject.toml
uv add prefect
```

To confirm that Prefect was installed successfully, run:

```
uv run prefect version
```


PREFECT: Flows, Tasks e States

- **Flow:** entrada do pipeline (como `@dag` no Airflow)
- **Task:** unidade de trabalho (como `@task` no Airflow)
- **States:** Pending, Running, Completed, Failed, Cached — rastreamento automático

```
from prefect import flow

@flow(log_prints=True)
def explain_flows():
    print("run any python code here!")
    print("encapsulate that business logic!")

if __name__ == "__main__":
    explain_flows()
```

```
from prefect import task

@task(log_prints=True)
def explain_tasks():
    print("run any python code here!")
    print("but maybe just a little bit")

if __name__ == "__main__":
    explain_tasks()
```

PREFECT: Flows, Tasks e States

- Cria o deployment + schedule e fica escutando

➤ `.../$ python example_1.py`

```
example_1.py U X
example_1.py > ...
1  from prefect import flow, task
2
3  @task
4  def fetch_data(url):
5      import requests
6      return requests.get(url).json()
7
8  @task
9  def process_data(dados):
10     return len(dados)
11
12  @flow
13  def meu_pipeline():
14     dados = fetch_data("https://fakestoreapi.com/products/1")
15     resultado = process_data(dados)
16     print(f"Processados: {resultado} registros")
17
18  if __name__ == "__main__":
19     meu_pipeline()
```

PROBLEMS OUTPUT TERMINAL PORTS POSTMAN CONSOLE AZURE QUERY RESULTS DEBUG CONSOLE

```
• (.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula 04/project_prefect$ python example_1.py
21:43:11.457 | INFO      | prefect - Starting temporary server on http://127.0.0.1:8230
See https://docs.prefect.io/v3/concepts/server#how-to-guides for more information on running a dedicated Prefect server.
21:43:14.294 | INFO      | Flow run 'truthful-shrew' - Beginning flow run 'truthful-shrew' for flow 'meu-pipeline'
21:43:15.348 | INFO      | Task run 'fetch_data-c8d' - Finished in state Completed()
21:43:15.357 | INFO      | Task run 'process_data-c2f' - Finished in state Completed()
Processados: 7 registros
21:43:16.323 | INFO      | Flow run 'truthful-shrew' - Finished in state Completed()
21:43:16.361 | INFO      | prefect - Stopping temporary server on http://127.0.0.1:8230
```


PREFECT: Flows, Tasks e States



PREFECT: Deployments e Work Pools

Deployment é a maneira de rodar seu flow agendado ou sob demanda. Há duas formas de execução:

A. `.server()`: Modo mais simples de execução.

- Uma linha no seu script: `meu_flow.serve(name="...", cron="0 8 * * *")`.
 - Cria o deployment, registra o schedule e inicia um processo que escuta runs.
 - Não precisa de work pool nem de worker.

B. `.deploy() + work pool + worker`: Modo de execução com infraestrutura dinâmica.

1. Você escreve o flow localmente (código)
2. Você publica com `.deploy()` apontando para um work pool.
3. Um worker escuta o work pool e executa (**Process**, **Docker**, **K8s**).
4. Você agenda via **UI** ou no `.deploy()`.

PREFECT: `.server()`

1. Iniciar o servidor do Prefect e configurar a variável de ambiente

```

PROBLEMS  OUTPUT  TERMINAL  PORTS  POSTMAN CONSOLE  AZURE  QUERY RESULTS  DEBUG CONSOLE

(.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula 04/project_prefect$ prefect server start

  _ _ _ _ _
 | _ \ _ \ _ | _ | _ / _ | _ | |
 | _ / _ / _ | _ | _ | ( _ | _ |
 | _ | _ \ _ | _ | _ \ _ | _ |

Configure Prefect to communicate with the server with:

    prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api

View the API reference documentation at http://127.0.0.1:4200/docs

Check out the dashboard at http://127.0.0.1:4200

```

```
PROBLEMS OUTPUT TERMINAL PORTS POSTMAN CONSOLE AZURE QUERY RESULTS DEBUG CONSOLE

(.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula 04/project_prefect$ prefect config set PREFECT_API_URL="http://127.0.0.1:4200/api"
Set 'PREFECT_API_URL' to 'http://127.0.0.1:4200/api'.
Updated profile 'local'.
(.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula 04/project_prefect$
```


PREFECT: .server()

2. Disparar manualmente o fluxo.

```
example_1.py U X
example_1.py > ...
13 def meu_pipeline():
14     dados = fetch_data("https://fakestoreapi.com/products/1")
15     resultado = process_data(dados)
16     print(f"Processados: {resultado} registros")
17
18 if __name__ == "__main__":
19     meu_pipeline.serve(
20         name="meu-deployment",
21         cron="1 * * * *",
22         tags=["exemplo_1"]
23     )
```

PROBLEMS OUTPUT TERMINAL PORTS POSTMAN CONSOLE AZURE QUERY RESULTS DEBUG CONSOLE python + v

```
o (.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula_04/project_prefect$ python example_1.py run "meu_pipeline/meu-deployment"
Your flow 'meu-pipeline' is being served and polling for scheduled runs!

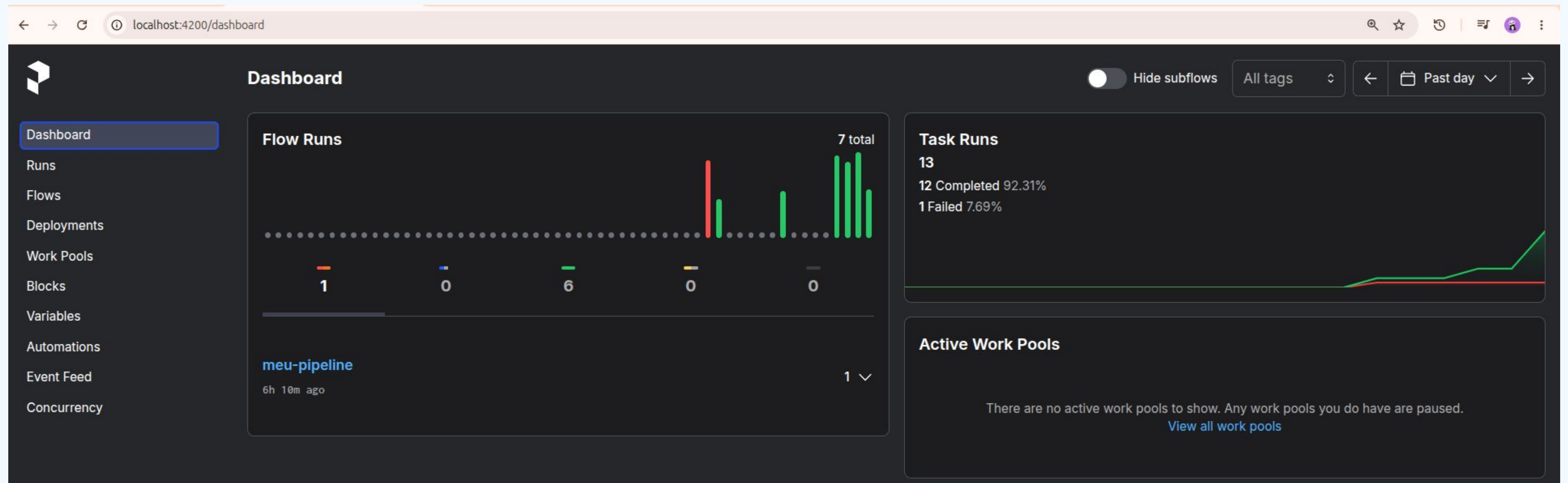
To trigger a run for this flow, use the following command:

    $ prefect deployment run 'meu-pipeline/meu-deployment'

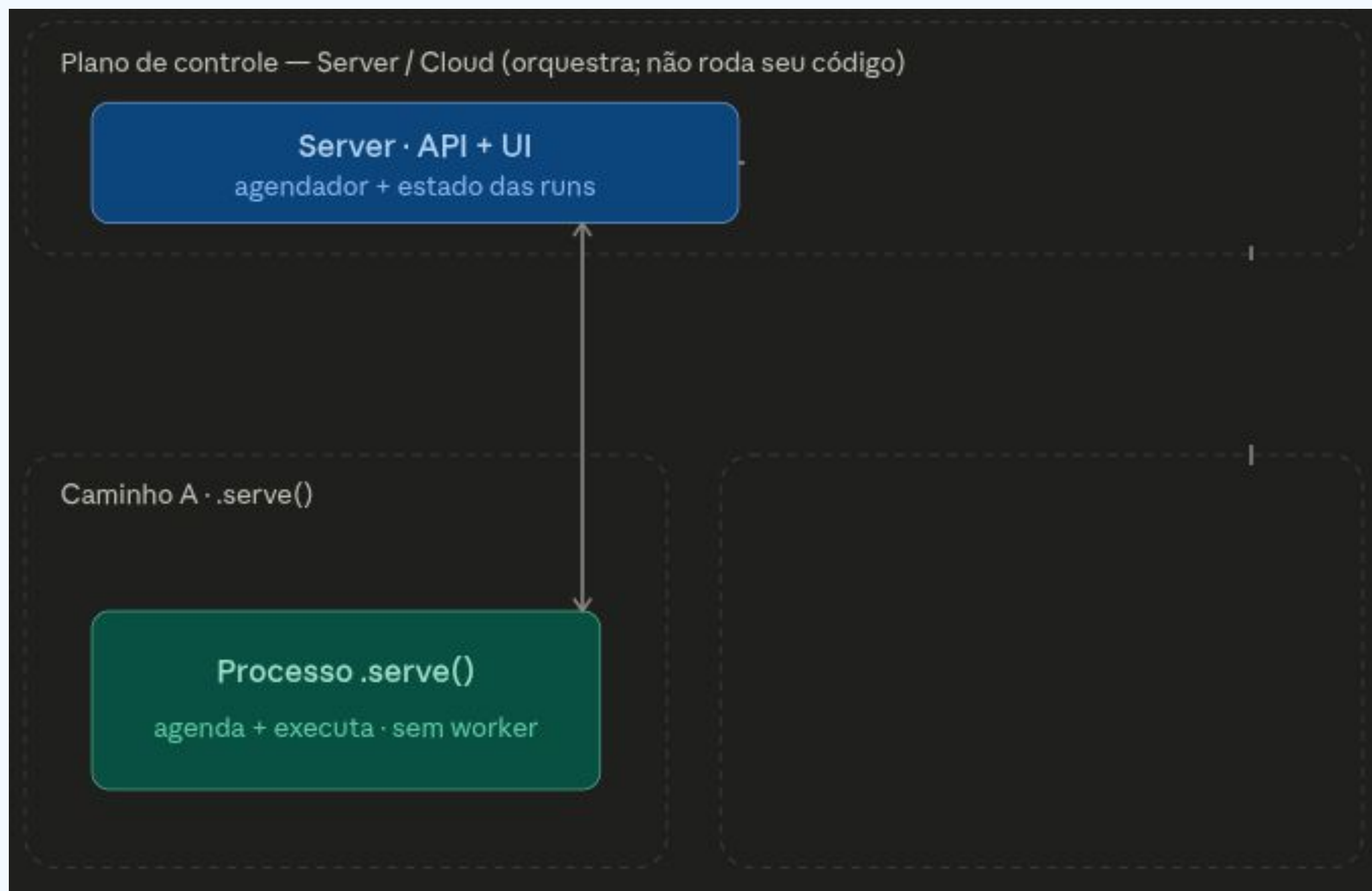
You can also run your flow via the Prefect UI: http://127.0.0.1:4200/deployments/deployment/d3bae494-dca4-4396-8479-fb18f3652305
```

PREFECT: Server API + UI

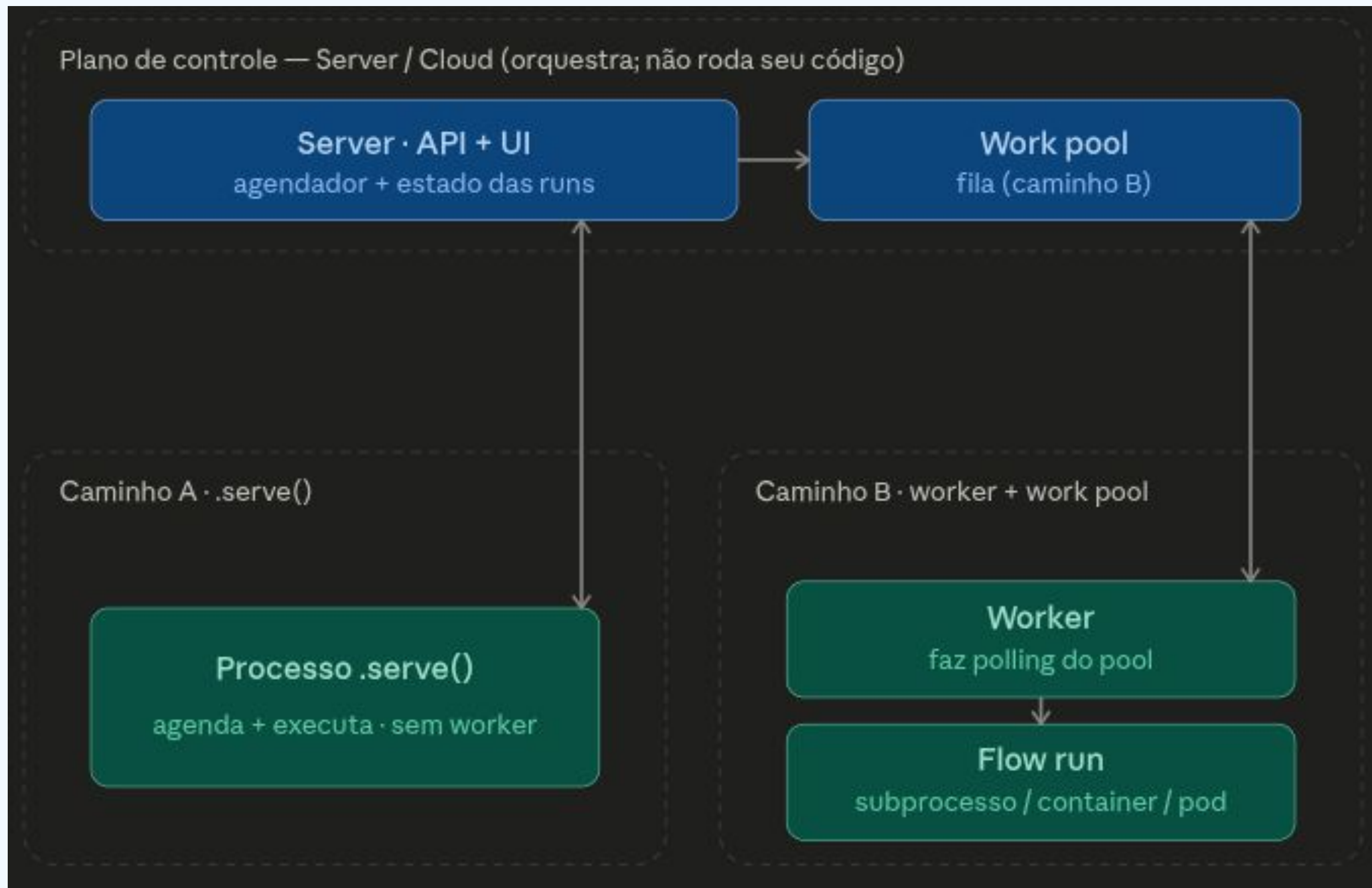
3. UI padrão do Prefect



PREFECT: Server API + UI



PREFECT: `.deploy()` + work pool + worker



PREFECT: `.deploy()` + work pool + worker

- ## 1. Iniciar o servidor do Prefect e configurar a variável de ambiente (realizado anteriormente)

[illegible]

```

PROBLEMS  OUTPUT  TERMINAL  PORTS  POSTMAN CONSOLE  AZURE  QUERY RESULTS  DEBUG CONSOLE

(.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula_04/project_prefect$ prefect config set PREFECT_API_URL="http://127.0.0.1:4200/api"
Set 'PREFECT_API_URL' to 'http://127.0.0.1:4200/api'.
Updated profile 'local'.
(.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula_04/project_prefect$

```

PREFECT: `.deploy()` + work pool + worker

2. Criar uma arquivo de deploy de flows. É possível utilizar diferentes formas, aqui vamos criar uma arquivo Python de deploy dentro da pasta do projeto.

```
.../$ deploy.py
```



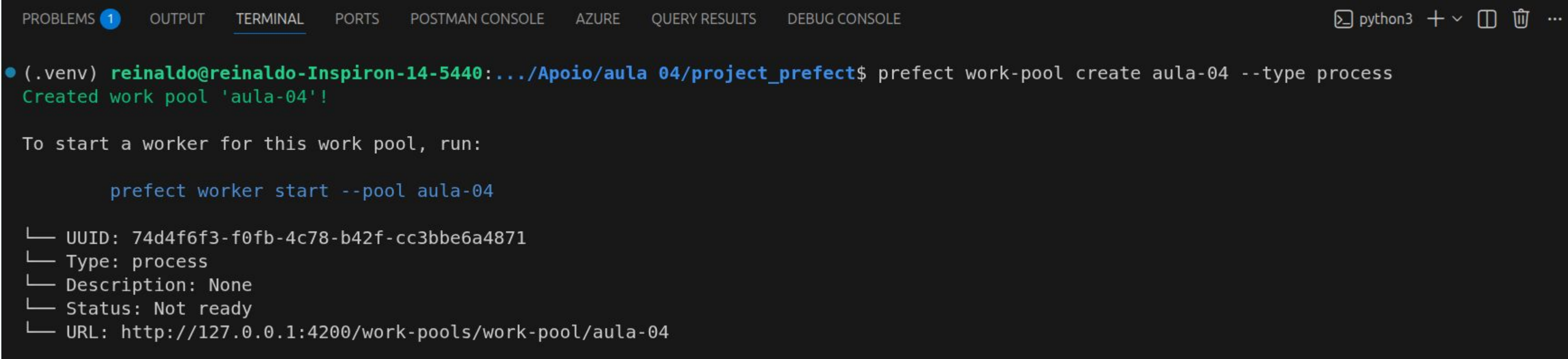
```
EXPLORER
...
PROJECT_PREFECT
  > __pycache__
  > .venv
  deploy.py 1, U
  example_1.py U
  requirements U

deploy.py 1, U
1  from prefect import flow
2
3  if __name__ == "__main__":
4      # Declaração do arquivo a ser deployado
5      flow.from_source(
6          source=".",                                # código local, mesma máquina do worker
7          entrypoint="example_1.py:meu_pipeline",
8      ).deploy(
9          name="exemplo-local",
10         work_pool_name="aula-04",
11         cron="0 20 * * *",                          # todo dia às 20:00
12     )
```


PREFECT: `.deploy()` + work pool + worker

3. Criar o work pool do tipo **process** chamado *aula-04*.

```
.../$ prefect work-pool create aula-04 --type process
```



```
PROBLEMS 1 OUTPUT TERMINAL PORTS POSTMAN CONSOLE AZURE QUERY RESULTS DEBUG CONSOLE python3 + v [ ] [ ] ...

• (.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula 04/project_prefect$ prefect work-pool create aula-04 --type process
Created work pool 'aula-04'!

To start a worker for this work pool, run:

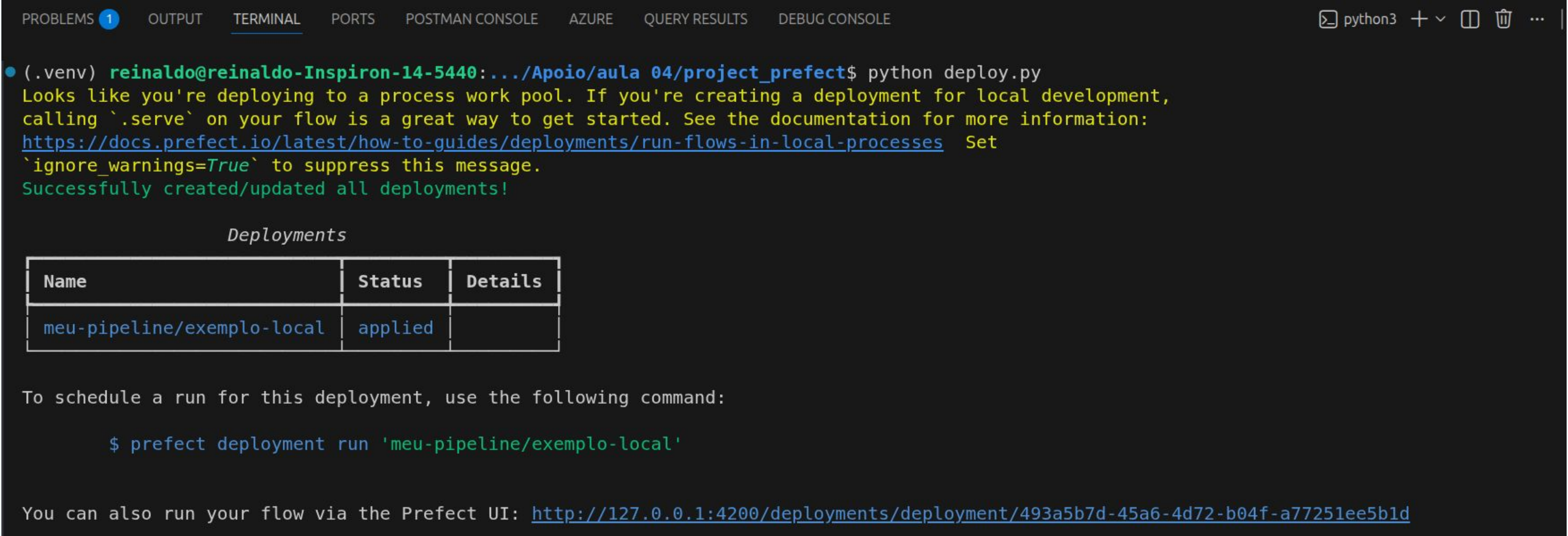
    prefect worker start --pool aula-04

└─ UUID: 74d4f6f3-f0fb-4c78-b42f-cc3bbe6a4871
└─ Type: process
└─ Description: None
└─ Status: Not ready
└─ URL: http://127.0.0.1:4200/work-pools/work-pool/aula-04
```

PREFECT: `.deploy()` + work pool + worker

4. Executar o arquivo de `deploy.py`

```
.../$ python deploy.py
```



```
PROBLEMS 1 OUTPUT TERMINAL PORTS POSTMAN CONSOLE AZURE QUERY RESULTS DEBUG CONSOLE
```

```
(.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula 04/project_prefect$ python deploy.py
```

Looks like you're deploying to a process work pool. If you're creating a deployment for local development, calling `.serve` on your flow is a great way to get started. See the documentation for more information: <https://docs.prefect.io/latest/how-to-guides/deployments/run-flows-in-local-processes> Set `'ignore_warnings=True'` to suppress this message.

Successfully created/updated all deployments!

Deployments

Name	Status	Details
meu-pipeline/exemplo-local	applied	

To schedule a run for this deployment, use the following command:

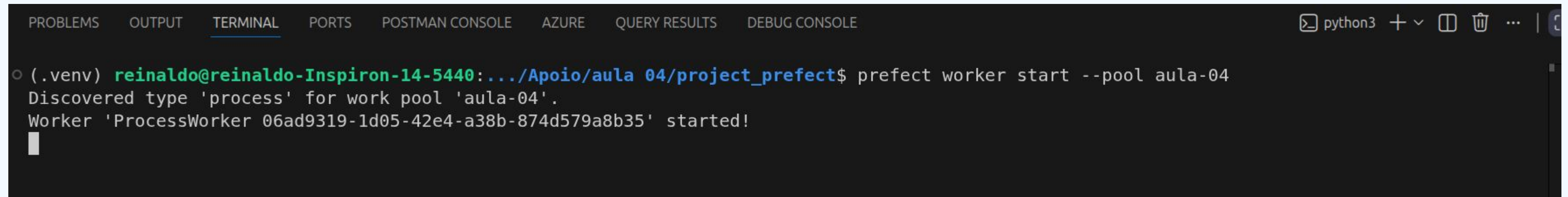
```
$ prefect deployment run 'meu-pipeline/exemplo-local'
```

You can also run your flow via the Prefect UI: <http://127.0.0.1:4200/deployments/deployment/493a5b7d-45a6-4d72-b04f-a77251ee5b1d>

PREFECT: `.deploy()` + work pool + worker

5. Iniciar o Worker

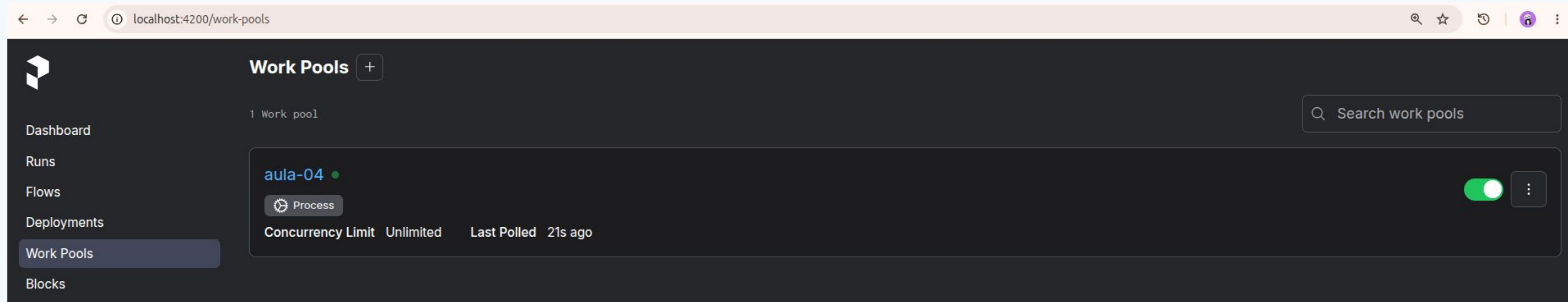
```
.../$ prefect worker start --pool aula-04
```



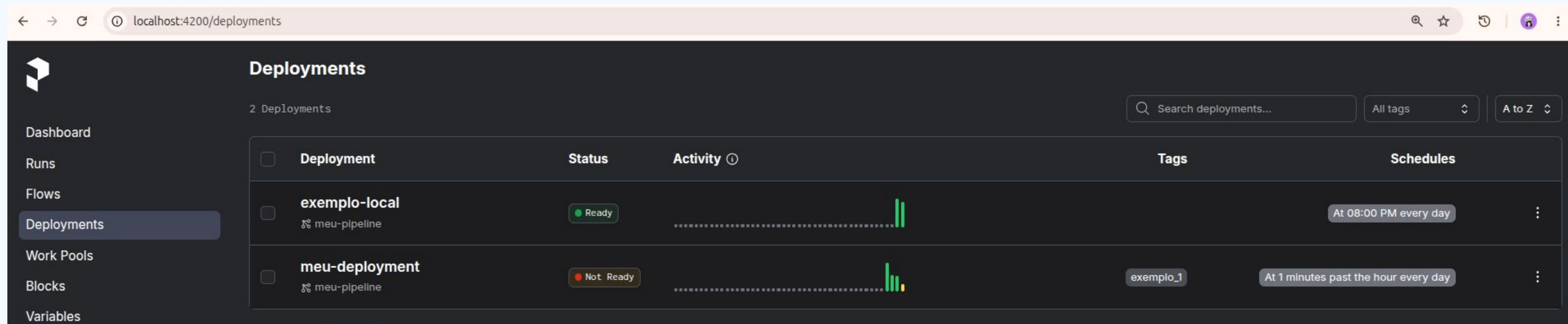
```
PROBLEMS OUTPUT TERMINAL PORTS POSTMAN CONSOLE AZURE QUERY RESULTS DEBUG CONSOLE python3 + v [ ] [ ] ... | [ ]
(.venv) reinaldo@reinaldo-Inspiron-14-5440:~/Apoio/aula_04/project_prefect$ prefect worker start --pool aula-04
Discovered type 'process' for work pool 'aula-04'.
Worker 'ProcessWorker 06ad9319-1d05-42e4-a38b-874d579a8b35' started!
```


PREFECT: `.deploy()` + work pool + worker

6. UI do Prefect, mostrando o work pool criado e o arquivo deployado.



The screenshot shows the Prefect UI 'Work Pools' page at `localhost:4200/work-pools`. The left sidebar contains links to Dashboard, Runs, Flows, Deployments, Work Pools (selected), and Blocks. The main content area shows '1 Work pool' with a search bar. A single work pool named 'aula-04' is listed with a green status dot, a 'Process' icon, and a toggle switch. Below the name, it shows 'Concurrency Limit: Unlimited' and 'Last Polled: 21s ago'.



The screenshot shows the Prefect UI 'Deployments' page at `localhost:4200/deployments`. The left sidebar contains links to Dashboard, Runs, Flows, Deployments (selected), Work Pools, Blocks, and Variables. The main content area shows '2 Deployments' with a search bar, 'All tags' dropdown, and 'A to Z' sort dropdown. A table lists the deployments:

Deployment	Status	Activity	Tags	Schedules
☐ exemplo-local meu-pipeline	Ready			At 08:00 PM every day
☐ meu-deployment meu-pipeline	Not Ready		exemplo_1	At 1 minutes past the hour every day

PREFECT: Blocks - Secrets e Config

É uma forma de armazenar e reutilizar configuração, segredos e credenciais

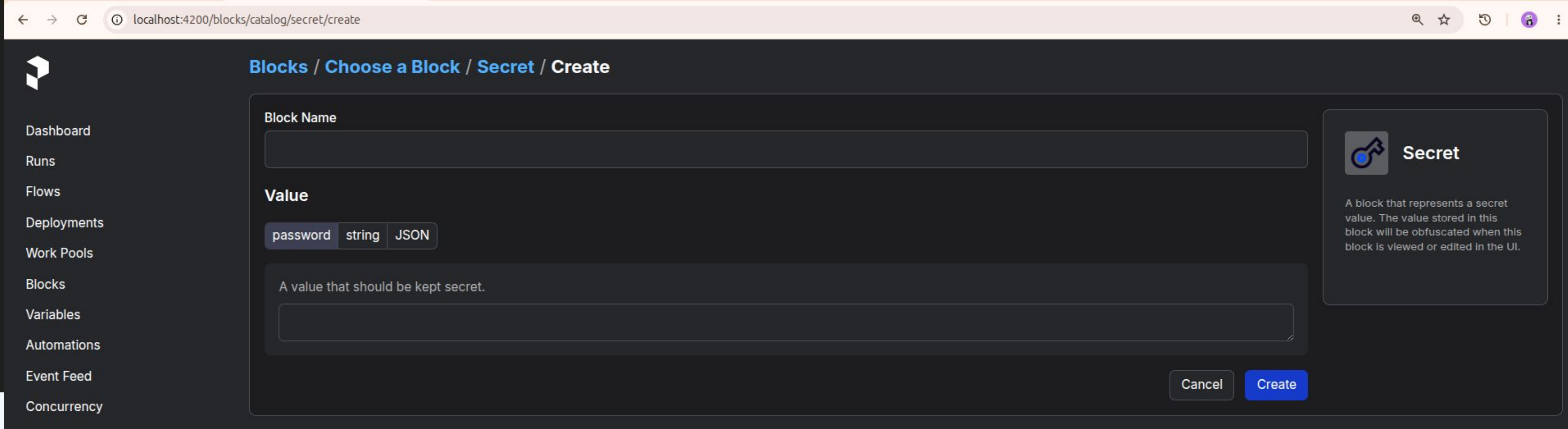
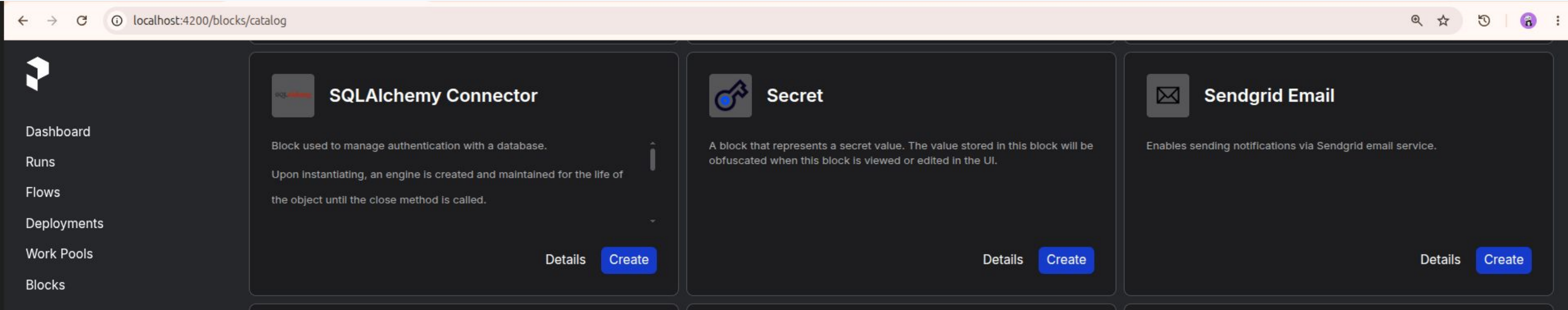
```
from prefect.blocks.system import Secret  
api_key = Secret.load("minha-api-key").get()
```

- Criar um Block na **UI** (*Blocks* → *Add Block*) ou via código
- Referenciar por nome em qualquer flow
- Mudou a senha? Atualiza no Block, não no código
- Secret vem no core.
 - Para banco, use a integração prefect-sqlalchemy (SqlAlchemyConnector): `pip install prefect-sqlalchemy`

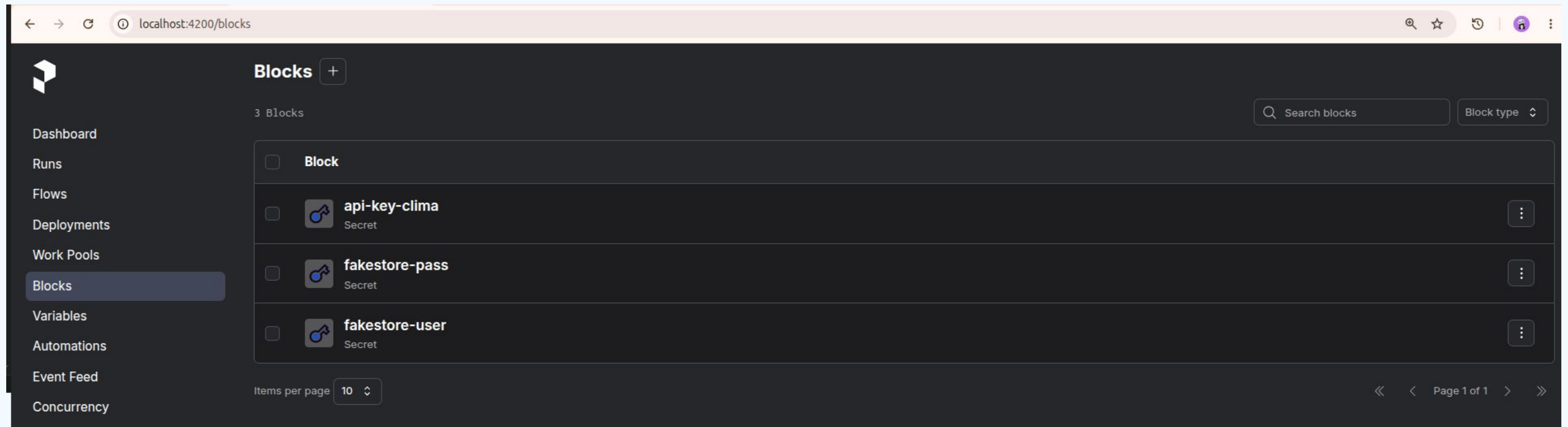
PREFECT: Blocks - Secrets e Config

[illegible]

PREFECT: Blocks - Secrets e Config



PREFECT: Blocks - Secrets e Config



Executar os arquivos:

`[example_2_block.py, example_3_block.py]`

- Atualize o arquivo `deploy.py` com os novos flows.
- Para a execução do Worker.
- Execute o arquivo `deploy.py` e na sequência start novamente o worker.

PREFECT: Map para paralelismo dinâmico

Um pool é um limite de recursos compartilhados. Você cria no Airflow:

- `.map()` cria uma task run por item em runtime (fan-out)
- Por padrão rodam de forma concorrente (`ThreadPoolTaskRunner = threads`)
- Equivale ao `.expand()` do Airflow - a diferença é a ergonomia.

*A sintaxe é enxuta, mas atenção ao vocabulário: `.map()` é concorrente por padrão (**threads**), ótimo para I/O como chamadas de API. Não é paralelismo de CPU — para isso troca-se o task runner (ProcessPool/Dask/Ray).*

PREFECT: Automations - event-driven workflows

Prefect Automations disparam workflows baseado em eventos, exemplo:

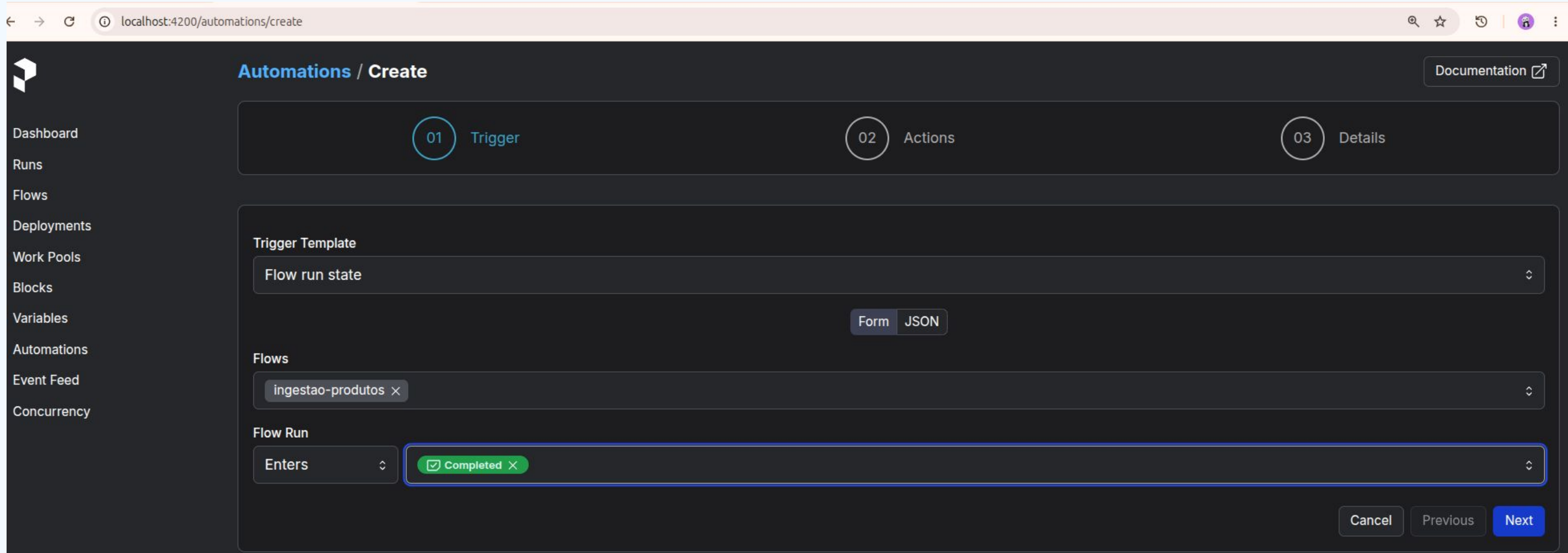
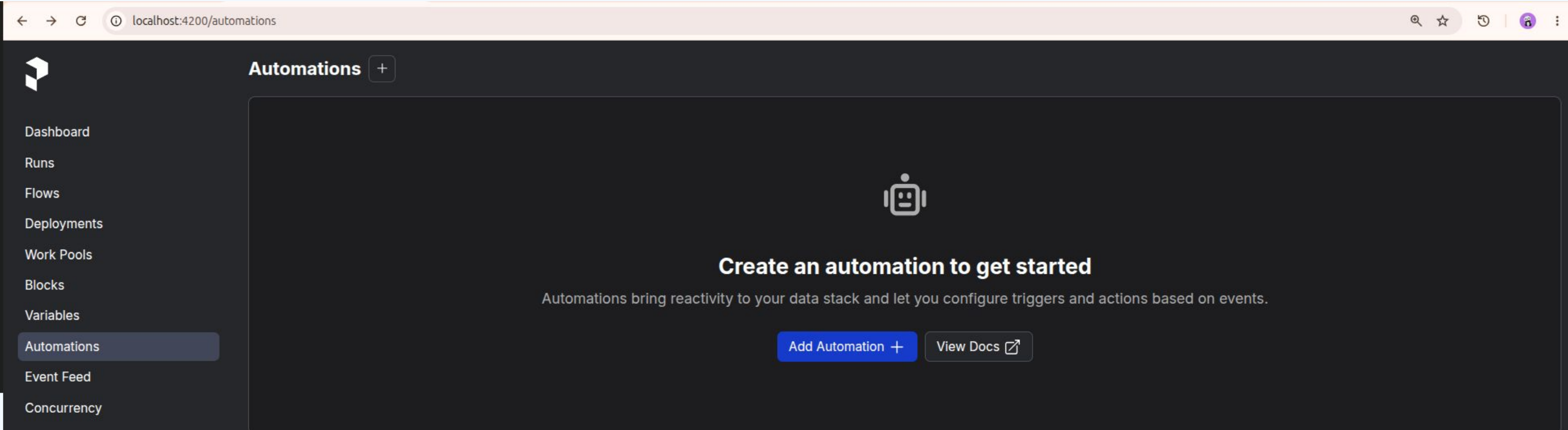
- Quando outro **flow** termina → dispara este
- Quando arquivo chega em **S3** → dispara este
- Quando **webhook** é chamado → dispara este
- A cada hora (**agendado**) → dispara este

Sem código — tudo configurável na UI:

- Admin → Automations → Create
- Trigger (quando?), Actions (o que fazer)

*Esta é uma grande vantagem do Prefect: Automations são **first-class citizens**. No **Airflow**, você precisa de `ExternalTaskSensor` ou `TriggerDagRun`. Aqui, é visual e simples.*

PREFECT: Automations - event-driven workflows



PREFECT: Automations - event-driven workflows

Dashboard

Runs

Flows

Deployments

Work Pools

Blocks

Variables

Automations

Event Feed

Concurrency

Automations / Create

Documentation

✓ Trigger

02 Actions

03 Details

Action 1

Action Type

Run a deployment

Deployment To Run

gerar-relatorio > exemplo-automacao-relatorio

Parameters

Parameters defined for an automation action will be merged with the deployment's default parameters when the action is executed.

Job Variables (Optional)

1 {}

2

3

Format

+ Add Action

Cancel

Previous

Next

PREFECT: Automations - event-driven workflows

Dashboard

Runs

Flows

Deployments

Work Pools

Blocks

Variables

Automations

Event Feed

Concurrency

Automations / Create

Documentation

✓ Trigger

✓ Actions

03 Details

Automation Name

Dispara Relatório

Description (Optional)

Cria um novo relatório ao fim da entrada de dados

Cancel

Previous

Save

Automations +

Documentation

1 automation

Dispara Relatório

Cria um novo relatório ao fim da entrada de dados

Trigger

When any flow run of flow ingestao-produtos enters

Completed

Action

Run deployment: exemplo-automacao-relatorio

Show parameters

Show job variables

PREFECT: Integrations - o ecossistema

Prefect tem 100+ integrations prontas:

- Cloud: AWS, GCP, Azure
- Data: dbt, Snowflake, BigQuery
- Messaging: Slack, PagerDuty
- File storage: S3, GCS, SFTP

Usar uma integração:

```
from prefect_aws.s3 import s3_download
```

- Instala como pacote: `pip install prefect-aws`
- Usa como qualquer função Python

O ecossistema do Prefect cresce rápido. Procure a integration que você precisa — provavelmente existe.

PREFECT: Logging e observabilidade

Prefect rastreia TUDO automaticamente:

- Logs: `print()` aparece nos logs da task
- Artifacts: salva outputs de tasks (CSV, JSON, plots)
- Traces: timeline detalhada da execução
- States: estado de cada task em cada momento

UI mostra:

- Flow runs: histórico de execuções
- Task runs: detalhes de cada task
- Logs viewer: logs em tempo real

A UI do Prefect é bem mais bonita e intuitiva que Airflow.

PREFECT: Prefect vs Airflow

Prefect vs Airflow — quando escolher cada um

Prefect

escolha se:

- ✓ ama Python puro, sem DSL
- ✓ simplicidade sobre flexibilidade
- ✓ quer event-driven nativo
- ✓ está começando — curva suave

Airflow

escolha se:

- ✓ já tem 100+ DAGs em produção
- ✓ precisa de extensibilidade máxima
- ✓ sua empresa já usa Airflow
- ✓ complexidade OK por mais poder

A escolha não é "qual é melhor" — é o contexto.

Airflow: mais maduro e poderoso. Prefect: mais moderno e direto.

Laboratório

- **ATIVIDADE 03 - PREFECT**